

Category and TravelPackage Management System

Scenario

The Travel Booking Management System is a database system used to manage different types of travel packages. The packages are organized into categories such as Domestic, International, Adventure, Honeymoon, Family, and Pilgrimage.

The database stores important information about each package, including the package ID, agency ID, category ID, package name, price, and available slots.

The main purpose of this project is to create tables, establish relationships, apply constraints, insert records, perform CRUD operations, and generate category-wise reports.

Create Category table

The CREATE TABLE command is used to create a new table in the database.

The **Category** table stores information about different types of travel packages. It contains the category ID, category name, and category description.

```
SQL> create table category(category_id number, category_name varchar(50), category_description varchar(200));

Table created.
```

TravelPackage Table

The TravelPackage table stores the details of travel packages available in the travel booking system.

This table contains the package ID, agency ID, category ID, package name, price, and number of available slots. It is connected to the Category table using Category_ID.

```
SQL> create table travelpackage(package_id number, agency_id number, category_id number, package_name varchar(100), price number(10,2), available_slots number);

Table created.
```

Describe the Table Structure

The DESCRIBE command is used to view the structure of a table. It displays the column names, data types, and other structural information.

```
SQL> desc category;
Name                               Null?    Type
-----
CATEGORY_ID                       NUMBER
CATEGORY_NAME                     VARCHAR2(50)
CATEGORY_DESCRIPTION               VARCHAR2(200)
```

```
SQL> desc travelpackage;
```

Name	Null?	Type
-----	-----	-----
PACKAGE_ID		NUMBER
AGENCY_ID		NUMBER
CATEGORY_ID		NUMBER
PACKAGE_NAME		VARCHAR2(100)
PRICE		NUMBER(10,2)
AVAILABLE_SLOTS		NUMBER

Define Relationships and Constraints

A category can contain many travel packages. Therefore, the relationship between Category and TravelPackage is One-to-Many (1 to *).

Primary Key Constraint

A Primary Key uniquely identifies each record in a table. Category_ID is the primary key of Category, and Package_ID is the primary key of TravelPackage.

```
SQL> alter table category add constraint pk_category primary key(category_id);
```

```
Table altered.
```

```
SQL> alter table travelpackage add constraint pk_travelpackage primary key(package_id);
```

```
Table altered.
```

Foreign Key Constraint

A Foreign Key is used to establish a relationship between two tables. Category_ID in TravelPackage refers to Category_ID in Category.

The Category_ID in the TravelPackage table refers to the Category_ID in the Category table.

This ensures that a travel package can only be assigned to an existing category.

```
SQL> ALTER TABLE TravelPackage
2  ADD CONSTRAINT fk_travelpackage_category
3  FOREIGN KEY (Category_ID)
4  REFERENCES Category(Category_ID);
```

```
Table altered.
```

NOT NULL Constraint

The NOT NULL constraint prevents a column from containing an empty value

This ensures that every category and every travel package must have a name.

```
SQL> alter table category modify category_name varchar(50) not null;
Table altered.

SQL> alter table travelpackage modify package_name varchar2(100) not null;
Table altered.
```

9. UNIQUE Constraint

The UNIQUE constraint prevents duplicate values from being stored in a column.

This prevents two categories from having the same category name and prevents two travel packages from having the same package name.

```
SQL> alter table category add constraint uq_category_name unique(category_name);
Table altered.

SQL> alter table travelpackage add constraint uq_package_name unique(package_name);
Table altered.
```

CHECK Constraint

The CHECK constraint ensures that the values entered into a column satisfy a particular condition

```
SQL> alter table category add constraint chk_category_id check(category_id > 0);
Table altered.

SQL> alter table travelpackage add constraint chk_package_price check(price > 0);
Table altered.

SQL> alter table travelpackage add constraint chk_available_slots check(available_slots >= 0);
Table altered.
```

Insert Values

The INSERT INTO command is used to add new records to a table.

```
SQL> insert into category values(1,'Domestic', 'Travel packages within the c  
ountry');
```

```
1 row created.
```

```
SQL> insert into category values(2,'International', 'Travel packages to fore  
ign country');
```

```
1 row created.
```

```
SQL> insert into category values(3,'honeymoon', 'Advdventure and outdoor trav  
el packages');
```

```
1 row created.
```

```
SQL> INSERT INTO Category  
2 VALUES (4, 'Honeymoon', 'Special honeymoon travel packages');
```

```
1 row created.
```

```
SQL> INSERT INTO Category  
2 VALUES (5, 'Family', 'Travel packages suitable for families');
```

```
1 row created.
```

```
SQL>
```

```
SQL> INSERT INTO Category  
2 VALUES (6, 'Pilgrimage', 'Religious and spiritual travel packages');
```

```
1 row created.
```

Travel package

1 row created.

```
SQL> INSERT INTO TravelPackage  
2 VALUES (101, 201, 1, 'Ooty Holiday Package', 12000, 20);
```

1 row created.

```
SQL> INSERT INTO TravelPackage  
2 VALUES (103, 202, 2, 'Dubai Holiday Package', 55000, 10);
```

1 row created.

```
SQL> INSERT INTO TravelPackage  
2 VALUES (104, 202, 2, 'Singapore Tour Package', 65000, 12);
```

1 row created.

```
SQL> INSERT INTO TravelPackage  
2 VALUES (105, 203, 3, 'Manali Adventure Trip', 25000, 18);
```

1 row created.

```
SQL> INSERT INTO TravelPackage  
2 VALUES (106, 203, 3, 'Rishikesh Adventure Tour', 22000, 14);
```

1 row created.

```
SQL> INSERT INTO TravelPackage  
2 VALUES (107, 204, 4, 'Goa Honeymoon Package', 30000, 10);
```

1 row created.

```
SQL> INSERT INTO TravelPackage  
2 VALUES (108, 204, 4, 'Kashmir Honeymoon Package', 45000, 8);
```

1 row created.

```
SQL> INSERT INTO TravelPackage  
2 VALUES (109, 205, 5, 'Mysore Family Package', 15000, 20);
```

1 row created.

```
SQL> INSERT INTO TravelPackage  
2 VALUES (110, 205, 5, 'Rajasthan Family Tour', 28000, 16);
```

1 row created.

```
SQL> INSERT INTO TravelPackage  
2 VALUES (111, 206, 6, 'Tirupati Pilgrimage Tour', 9000, 25);
```

1 row created.

```
SQL> INSERT INTO TravelPackage  
2 VALUES (112, 206, 6, 'Varanasi Spiritual Tour', 14000, 20);
```

Display Records

The SELECT command is used to retrieve records from a table.

The SELECT * command displays all columns and all records stored in the respective table.

```
SQL> select*from category;
```

```
CATEGORY_ID CATEGORY_NAME
```

```
-----  
CATEGORY_DESCRIPTION
```

```
-----  
          1 Domestic  
Travel packages within the country
```

```
          2 International  
Travel packages to foreign country
```

```
          3 honeymoon  
Adventure and outdoor travel packages
```

```
CATEGORY_ID CATEGORY_NAME
```

```
-----  
CATEGORY_DESCRIPTION
```

```
-----  
          4 Honeymoon  
Special honeymoon travel packages
```

```
          5 Family  
Travel packages suitable for families
```

```
          6 Pilgrimage  
Religious and spiritual travel packages
```

```
6 rows selected.
```

```
SQL> |
```

```
SQL> select*from travelpackage;
```

```
PACKAGE_ID  AGENCY_ID  CATEGORY_ID
```

```
PACKAGE_NAME
```

```
PRICE  AVAILABLE_SLOTS
```

```
101      201      1
Ooty Holiday Package
12000      20
```

```
103      202      2
Dubai Holiday Package
55000      10
```

```
PACKAGE_ID  AGENCY_ID  CATEGORY_ID
```

```
PACKAGE_NAME
```

```
PRICE  AVAILABLE_SLOTS
```

```
104      202      2
Singapore Tour Package
65000      12
```

```
105      203      3
Manali Adventure Trip
```

```
PACKAGE_ID  AGENCY_ID  CATEGORY_ID
```

```
PACKAGE_NAME
```

```
PRICE  AVAILABLE_SLOTS
```

```
25000      18
```

```
106      203      3
Rishikesh Adventure Tour
22000      14
```

```
107      204      4
```

```
PACKAGE_ID  AGENCY_ID  CATEGORY_ID
```

```
PACKAGE_NAME
```

```
PRICE  AVAILABLE_SLOTS
```

```
Goa Honeymoon Package
30000          10
```

```
108          204          4
```

```
Kashmir Honeymoon Package
45000          8
```

```
PACKAGE_ID  AGENCY_ID  CATEGORY_ID
```

```
PACKAGE_NAME
```

```
PRICE  AVAILABLE_SLOTS
```

```
109          205          5
Mysore Family Package
15000          20
```

```
110          205          5
Rajasthan Family Tour
28000          16
```

```
PACKAGE_ID  AGENCY_ID  CATEGORY_ID
```

```
PACKAGE_NAME
```

```
PRICE  AVAILABLE_SLOTS
```

```
111          206          6
Tirupati Pilgrimage Tour
9000          25
```

```
112          206          6
Varanasi Spiritual Tour
```

```
Varanasi Spiritual Tour
```

```
PACKAGE_ID  AGENCY_ID  CATEGORY_ID
```

```
PACKAGE_NAME
```

```
PRICE  AVAILABLE_SLOTS
```

```
14000          20
```

```
11 rows selected.
```

CRUD Operations

CRUD stands for:

- C – Create

- **R – Read**
- **U – Update**
- **D – Delete**

These are the basic operations performed on database records.

Update Record and delete record

The UPDATE command is used to modify an existing record.

This changes the price of the travel package whose package ID is 101 to ₹13,000. The WHERE condition identifies which particular record should be updated.

The DELETE command is used to remove a record from a table.

This deletes the travel package whose package ID is 113. The WHERE condition ensures that only the specified record is deleted.

```
SQL> INSERT INTO TravelPackage
  2  VALUES (113, 207, 1, 'Coorg Travel Package', 16000, 15);

1 row created.

SQL> UPDATE TravelPackage
  2  SET Price = 13000
  3  WHERE Package_ID = 101;

1 row updated.

SQL> DELETE FROM TravelPackage
  2  WHERE Package_ID = 113;

1 row deleted.
```

Category-wise Travel Package Report

This query displays travel packages according to their category ID. The ORDER BY clause arranges the records in category-wise order.

```
SQL> SELECT Category_ID,
  2          AVG(Price) AS Average_Price
  3  FROM TravelPackage
  4  GROUP BY Category_ID
  5  ORDER BY Category_ID;

CATEGORY_ID  AVERAGE_PRICE
-----
          1          13000
          2          60000
          3          23500
          4          37500
          5          21500
          6          11500

6 rows selected.
```

Count Number of Packages in Each Category

The COUNT() function calculates the number of travel packages in each category. The GROUP BY clause groups packages according to their category ID.

```
SQL> SELECT Category_ID,  
2          COUNT(*) AS Total_Packages  
3 FROM TravelPackage  
4 GROUP BY Category_ID  
5 ORDER BY Category_ID;
```

CATEGORY_ID	TOTAL_PACKAGES
1	1
2	2
3	2
4	2
5	2
6	2

6 rows selected.

Find Highest-Priced Package in Each Category

The MAX() function finds the highest package price within each category. This helps identify the most expensive travel package in every category.

```
SQL> SELECT Category_ID,  
2          MAX(Price) AS Highest_Price  
3 FROM TravelPackage  
4 GROUP BY Category_ID  
5 ORDER BY Category_ID;
```

CATEGORY_ID	HIGHEST_PRICE
1	13000
2	65000
3	25000
4	45000
5	28000
6	14000

6 rows selected.

Average Package Price Category-wise

The AVG() function calculates the average price of travel packages in each category. This provides a category-wise comparison of the average travel package price.

```
SQL> SELECT Category_ID,  
2         AVG(Price) AS Average_Price  
3 FROM TravelPackage  
4 GROUP BY Category_ID  
5 ORDER BY Category_ID;
```

CATEGORY_ID	AVERAGE_PRICE
1	13000
2	60000
3	23500
4	37500
5	21500
6	11500

6 rows selected.